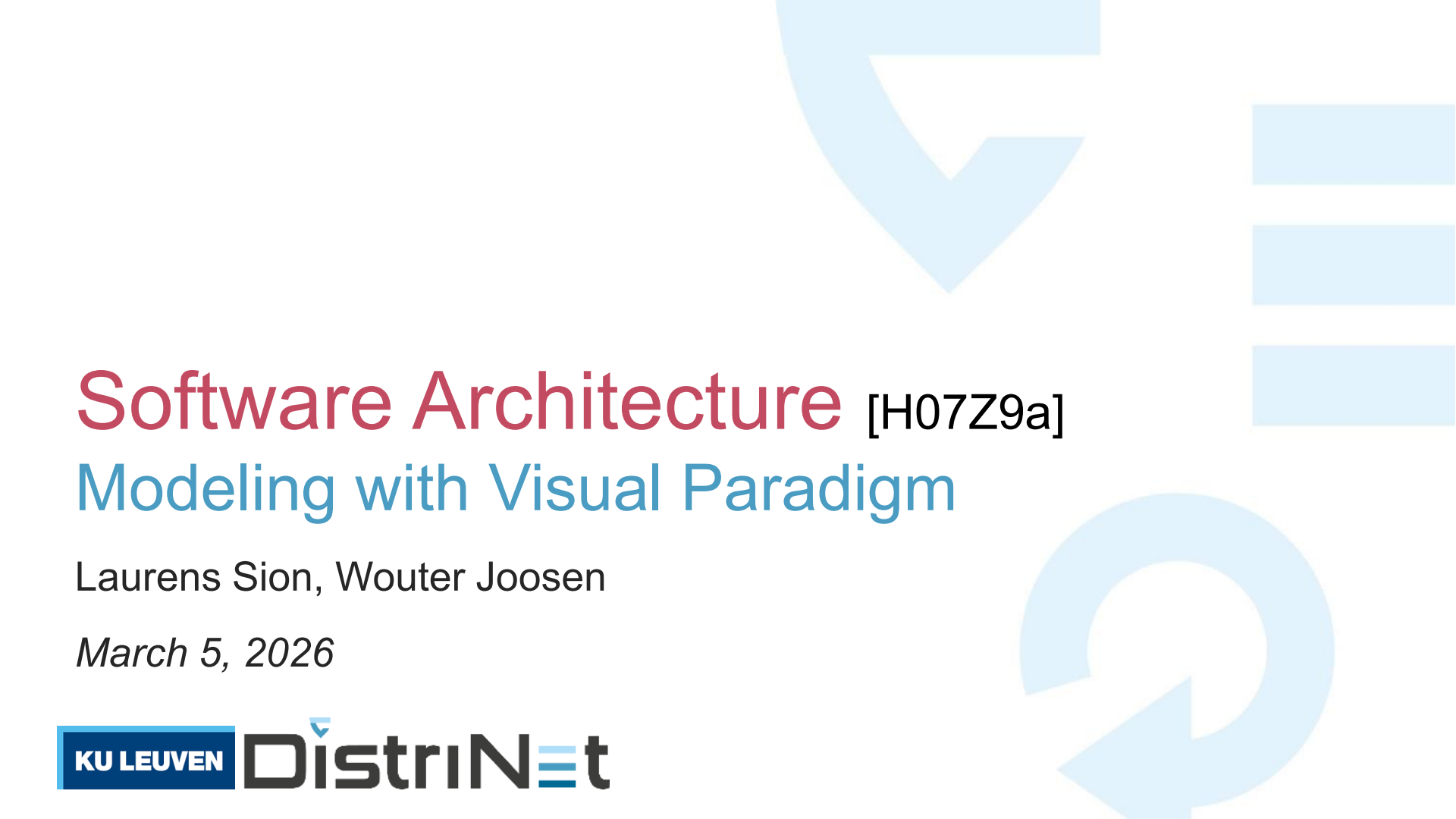




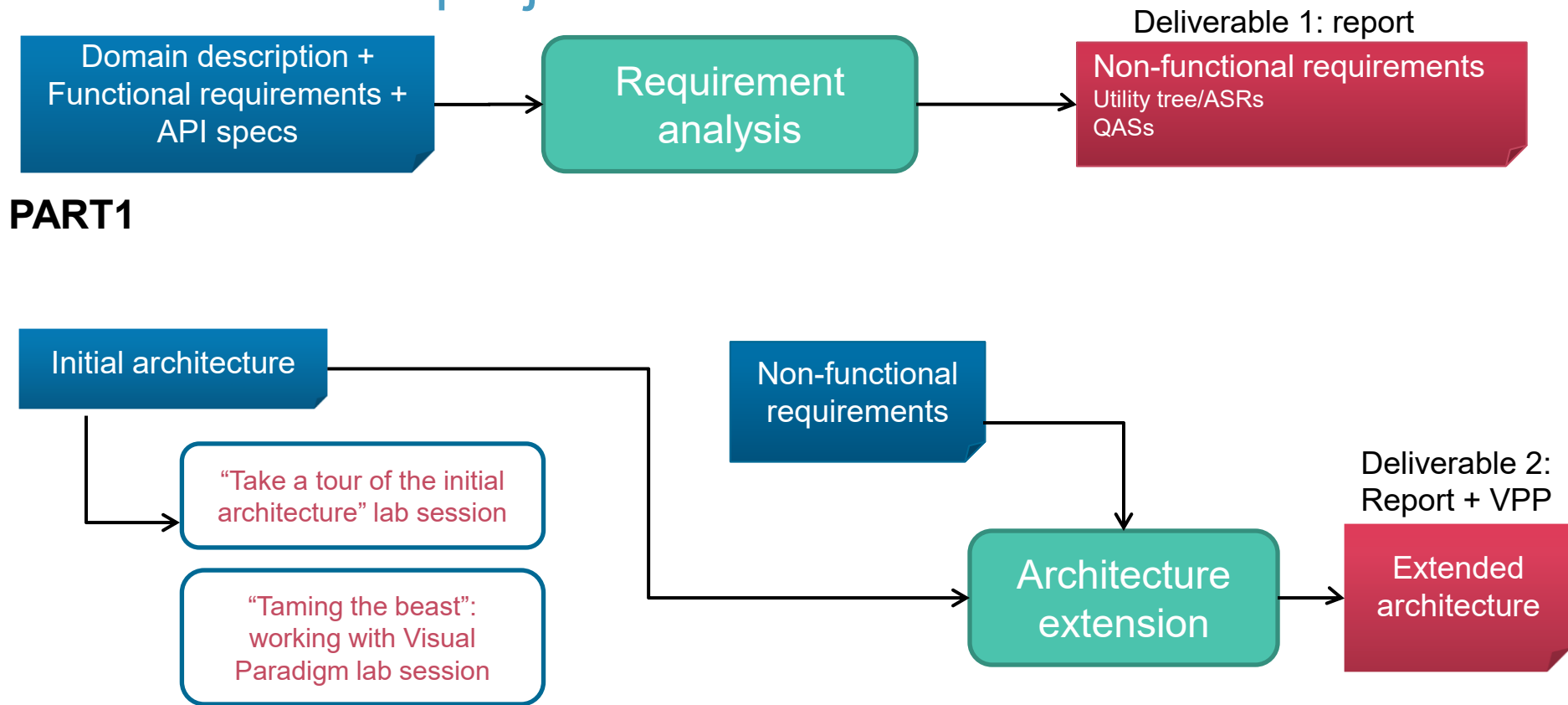
Software Architecture [H07Z9a]

Modeling with Visual Paradigm

Laurens Sion, Wouter Joosen

March 5, 2026

Timeline of the project

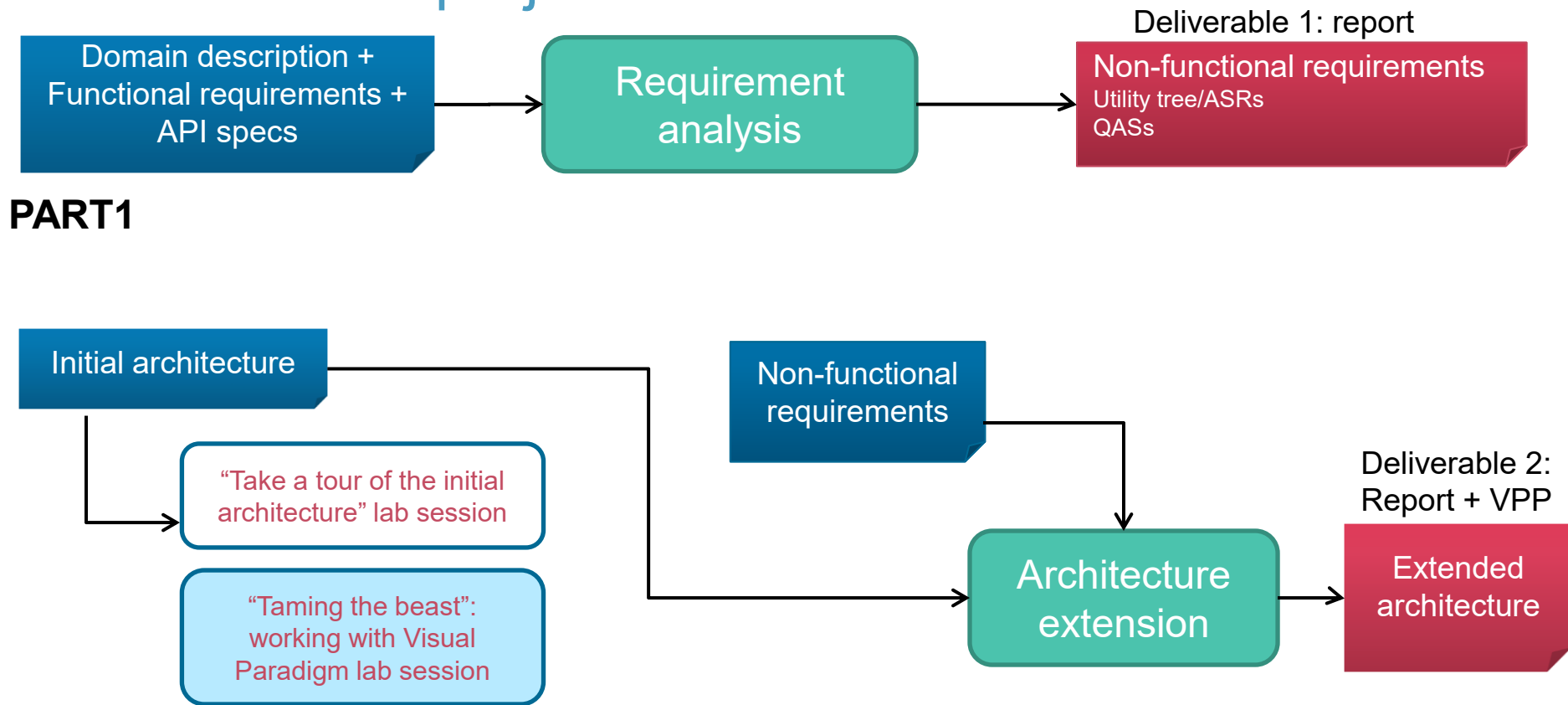


PART2

Continuous feedback

- › Part 1: Grade+ feedback + option to resubmit improved version
- › Part 2: SA Plugin for Visual Paradigm – *some* compiler comfort
- › Part 2: Opportunity to ask questions in all sessions (Lab sessions & plenary lectures)

Timeline of the project

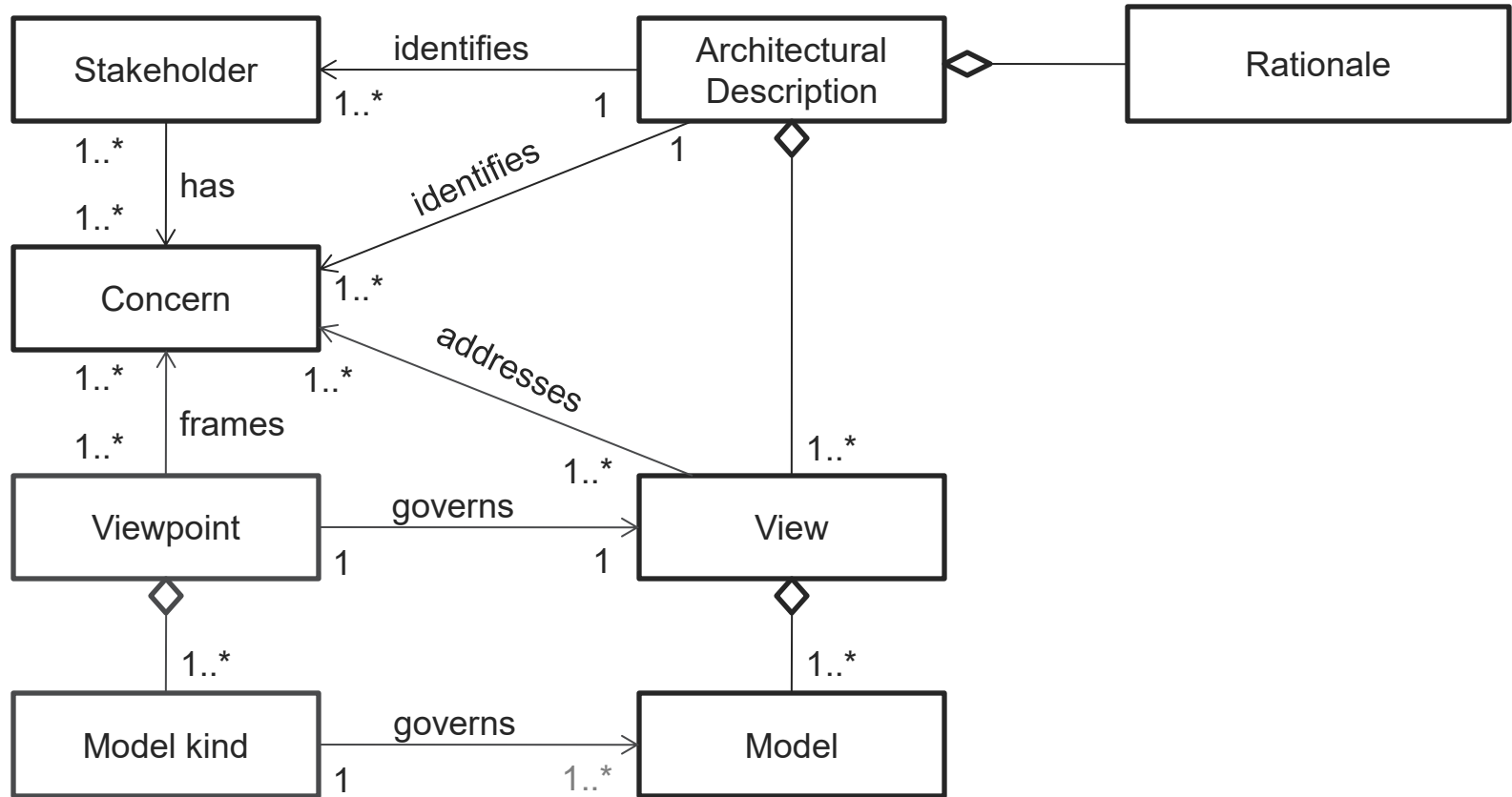


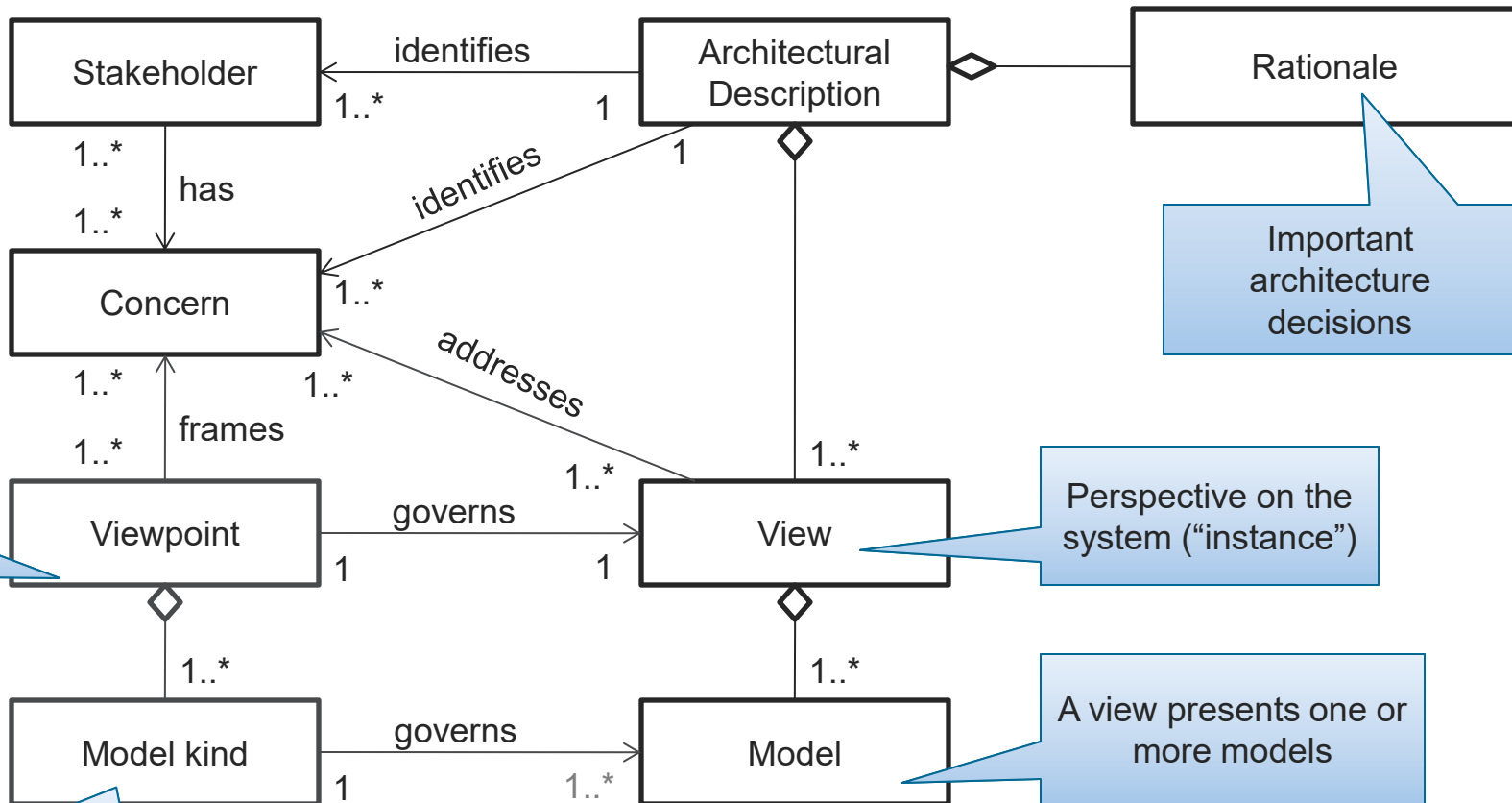
PART2

Intermezzo: Architecture in other disciplines

Terminology and concepts

ISO 42010 System and software engineering – Architecture description (2000/2011)





Important
architecture
decisions

Perspective on the
system ("instance")

A view presents one or
more models

"type"

e.g. UML Component
Diagram

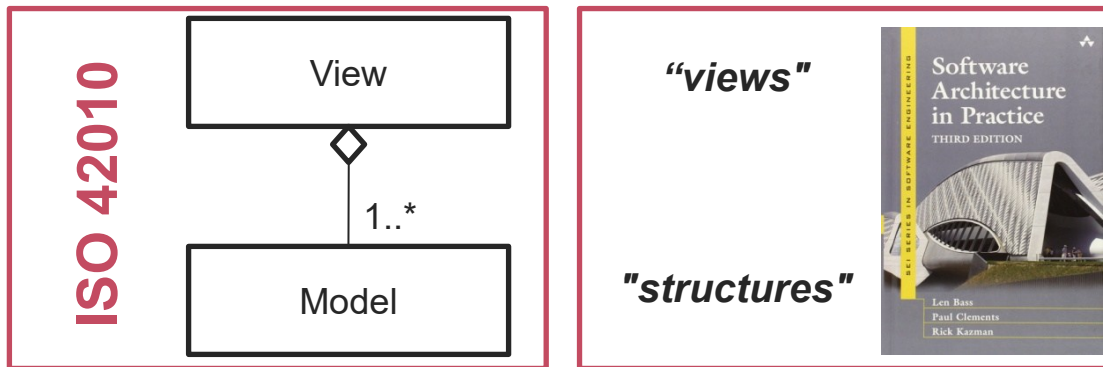
ISO 42010

Architecture view

- › An architecture view is a simplified description (**abstraction**) of a system
 - › From a particular **perspective** (stakeholders and concerns)
 - › Omitting entities that are not relevant to this perspective
- › Contains **models** addressing all the concerns
- › Is the result of applying a **viewpoint** (and refers to it)

Architectural viewpoints

- » **Component-and-connector viewpoints** (runtime, quality)
 - »» How the software is structured as a set of **elements that have runtime behavior and interactions**
- » **Module viewpoints** (structure, design, functionality)
 - »» How the software is structured as a set of **implemented units having responsibilities**
- » **Allocation viewpoints**
 - »» How the software **relates to non-software structures in the environment**



C&C viewpoint

› Elements

- › A **component** is an element with **runtime presence** (processes, clients, servers, and data stores)
 - ›› Components can have *ports*, i.e., points of (potential) interaction at runtime
- › A **connector** is an interaction means between the components (communication, information flows)
 - ›› the visibility between two parts
 - ›› a communication means

› Relations

- › Attachment of connectors to components to form a **graph**
 - ›› Who talks to who, and how

Module viewpoint

› Elements

- › A **module** is an **implementation unit** that provides a coherent set of responsibilities.
 - ›› E.g., a class, a collection of classes, a layer, an aspect, or any decomposition of the implementation unit.
- › Structural elements and their relationships

› Relations

- › Is part of → decomposition and aggregation (**our focus**)
- › Depends on → dependencies between modules
- › Is a → generalization/specialization

Allocation viewpoint in general

- › Elements

- › Software element and **environmental** element

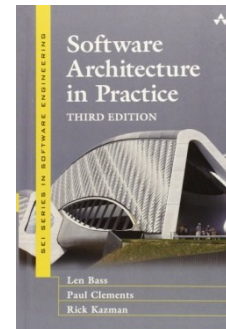
- › Relations

- › Allocated-to (distribution of software)

- › E.g. work assignment view (teams/developers), implementation (files), deployment view (machines, nodes)

In this course...

- › Terminology aligned to **ISO 42010**
- › **Selected architectural views** from **textbook**
 1. Client-server view (C&C)
 2. Decomposition view (Module),
 3. Process view (C&C) and
 4. Deployment view (Allocation)



(in function of the software qualities that we care about)

Client-server view (type: C&C)

Purpose

- › **Components** interact in a **client-server relation** (*uses*)
- › To answer the following questions
 - ›› What are the main data stores?
 - ›› What is the flow of control through the system?
 - ›› How does data progress through a system as it executes?
 - ›› What parts of the system run in parallel?
 - ›› How can the system's structure change as it executes?
- › Performance, reliability, functionality, security, ...

Process view (type: C&C)

Purpose

- › *Refining* the connectors
 - ›› Process, protocol, state-based behavior
- › Timing
 - ›› Sequential messages
 - ›› Synchronous/asynchronous messages
 - ›› Conditional behavior
 - ›› Loops
 - ›› Exceptions
 - ›› Relationships between inputs and outputs
- › *Performance, availability, functionality, security*

Decomposition view (type: module)

Purpose

› Communication

- › The various levels of granularity of the module decomposition provide a **top-down presentation** of the system's responsibilities

› Construction

- › Blueprint for the source code
- › Reuse of libraries
- › Modularity

› Modifiability, extensibility, interoperability, etc

Deployment view (type: allocation)

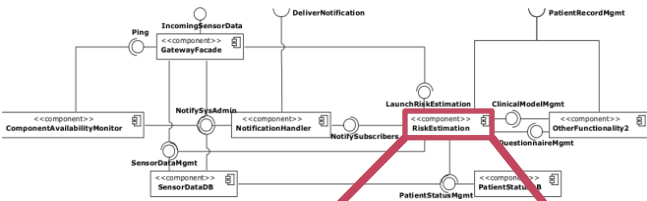
Purpose

- › Environmental elements
 - › Where software components reside **at run-time**
 - › Hardware (computers, processors, devices, peripherals), application servers, etc.
- › Identify the **distribution** architecture
 - › Do you intend to take a fat-client approach where the **business logic is contained in a desktop** application or a thin-client approach where business logic is deployed to an application server?
 - › Will your application have two **tiers**, three tiers, or more?
 - › What **HW and OS** are to be deployed?
 - › How will the various nodes be connected (perhaps via **RMI** and a **message bus**), protocols, etc
 - › **Replication** and multiple component instances
- › Important for Installation/configuration
- › **Availability, performance, portability, data consistency, ...**

Lab session on Monday + lecture next week

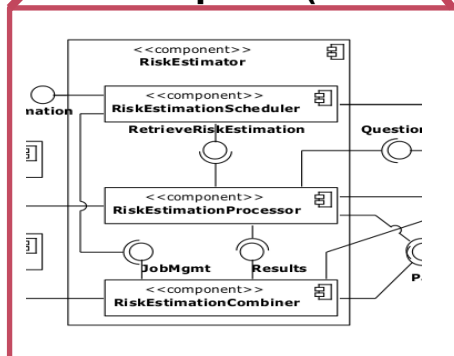
LOGICAL

Client-Server viewpoint (C&C)



UML Component diagram

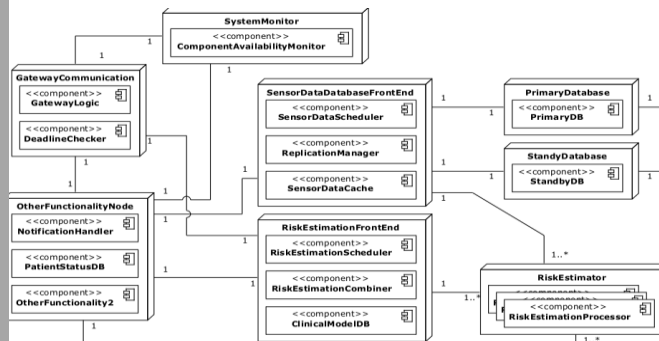
Decomposition viewpoint (Module)



UML Component diagram

PHYSICAL

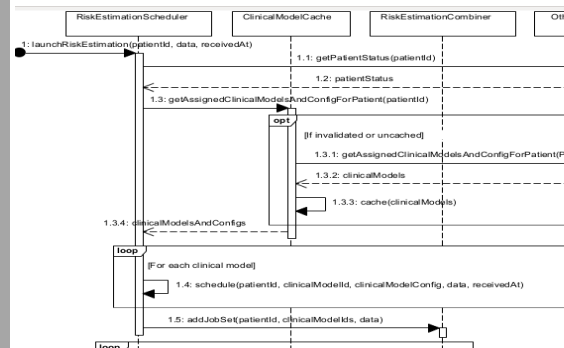
Deployment viewpoint (Allocation)



UML Deployment diagram

PROCESS

Process viewpoint (C&C)



UML Sequence diagram

LEGEND

RED: Kruchten architectural framework
Black: SAiP architectural framework
Blue: UML Diagram types

Context diagrams
 Element Catalog

An introduction to Visual Paradigm and the SAPlugin